Ricerca predittiva regolata
Un esperimento sul giro di consegne

Francesco Vampa
Matricola: 316461

26 ottobre 2026

Indice

Introduzione **2**

1 Ricerca **3**

2 Introduzione della previsione **6**

3 Origine dei target **7**

4 Ricerca predittiva non regolata **8**

5 Regolazione della ricerca predittiva **9**

6 Errore di previsione e necessità di ricerca **10**

7 Scelta probabilistica e massa irrisolta **11**

8 Esperimento **11**

9 Discussione **13**

10 Materiale di consegna e riproducibilità **14**

A Registro delle decisioni supportate da AI **16**

Riferimenti bibliografici **17**

Introduzione

Il problema del giro di consegne consiste nel determinare un percorso che permetta a un veicolo di partire da un deposito, visitare tutti i punti di consegna una sola volta e tornare al deposito, minimizzando la distanza totale percorsa. In questo lavoro, i luoghi sono rappresentati da punti su un piano e gli spostamenti sono valutati mediante distanze euclidee. Il problema corrisponde quindi al problema del commesso viaggiatore euclideo (*Euclidean Traveling Salesman Problem*, TSP euclideo).

Il numero dei possibili ordini di visita cresce rapidamente all'aumentare delle consegne, rendendo utile scegliere quali alternative esplorare quando il calcolo disponibile è limitato. L'obiettivo del progetto è valutare se un modello predittivo possa guidare la ricerca verso percorsi promettenti e se un regolatore possa migliorarne ulteriormente l'efficacia, decidendo dove dedicare ulteriore calcolo. La distinzione centrale è tra la previsione di quali alternative sembrano promettenti e la regolazione del calcolo dedicato a tali alternative. L'esperimento riportato confronta la ricerca di base, quella predittiva e quella predittiva regolata a budget di estrazioni comparabili. La regolazione modifica la scelta del prossimo stato quando la ricerca non migliora il giro corrente, distanziando gli interventi per conservare anche l'esplorazione guidata dal modello.

Complessità computazionale

Il TSP è un problema di ottimizzazione **NP-hard**, anche quando le distanze sono euclidee [1]. La classificazione riguarda la famiglia di istanze di dimensione variabile: non implica che ogni singola istanza sia difficile da risolvere. In particolare, istanze piccole possono essere risolte esattamente e utilizzate come riferimento negli esperimenti.

Per il TSP generale, la NP-hardness può essere dimostrata mediante una riduzione polinomiale dal problema del circuito hamiltoniano [2]. Dato un grafo $G = (V, E)$, si costruisce un grafo completo sugli stessi vertici, assegnando costo 0 agli archi presenti in E e costo 1 agli archi aggiunti. Il grafo originale contiene un circuito hamiltoniano se e solo se il grafo costruito ammette un giro TSP di costo 0. Un algoritmo esatto per il TSP permetterebbe quindi di decidere l'esistenza del circuito hamiltoniano. Quest'ultimo visita ogni *vertice* una sola volta, mentre un circuito euleriano attraversa ogni *arco* una sola volta.

Questa costruzione riguarda il TSP generale: i costi così assegnati non sono necessariamente rappresentabili come distanze euclidee. La NP-hardness della variante euclidea è un risultato distinto, richiamato in [1]. Nel presente progetto si mantiene questa variante e si studia la qualità delle soluzioni ottenute con risorse di ricerca limitate, senza pretendere di eliminare la difficoltà del problema nel caso generale.

1 Ricerca

In questa sezione viene descritta la ricerca di base adottata per il problema del giro del corriere. La procedura utilizza una visita in profondità (DFS) per determinare l'ordine di esplorazione degli stati, integrata con un meccanismo di branch-and-bound per scartare i rami che non possono migliorare la migliore soluzione completa già trovata.

1.1 Rappresentazione dello stato e spazio di ricerca

Prima di descrivere l'algoritmo, si definiscono i concetti di stato e spazio di ricerca.

Uno *stato* rappresenta una configurazione del problema in un dato momento della ricerca. Nel problema del giro del corriere, ciascuno stato contiene il percorso parziale costruito fino a quel momento, l'insieme delle consegne ancora da effettuare e la distanza accumulata. La posizione attuale del corriere corrisponde all'ultimo punto del percorso.

Lo stato iniziale contiene il solo deposito nel percorso, tutte le consegne ancora da effettuare e un costo pari a zero. Quando tutte le consegne sono state visitate, il giro viene completato tornando al deposito. La distanza di ritorno viene inclusa nel costo finale.

Lo *spazio di ricerca* è l'insieme degli stati raggiungibili dallo stato iniziale attraverso le azioni consentite. In questo caso, comprende i percorsi ammissibili che partono dal deposito e visitano un sottoinsieme delle consegne, senza ripetizioni. Gli stati che hanno visitato tutte le consegne costituiscono le foglie dell'albero di ricerca; a ciascuna foglia si associa un giro completo aggiungendo il ritorno al deposito.

1.2 Espansione e generazione dei successori

L'espansione di uno stato consiste nella generazione dei possibili proseguimenti del percorso corrente.

Sia $V = \{d, c_1, \ldots, c_n\}$ l'insieme delle località, dove d rappresenta il deposito e $c_1, \ldots, c_n$ i punti di consegna. Per ciascuno stato si distinguono l'insieme $C \subseteq V$ delle località già visitate, comprendente il deposito, e l'insieme $N = V \setminus C$ delle consegne ancora da effettuare. Si ha quindi:

$$C \cap N = \emptyset, \qquad C \cup N = V.$$

Lo stato viene indicato con $s = (p, N, g)$, dove p conserva l'ordine delle località visitate e g indica il costo accumulato. L'insieme C si ricava dal percorso p e non deve quindi essere memorizzato separatamente.

Per ogni consegna $v \in N$ viene generato uno stato successore. Il nuovo percorso si ottiene aggiungendo v alla fine di p, mentre gli insiemi vengono aggiornati come:

$$C' = C \cup \{v\}, \qquad N' = N \setminus \{v\}.$$

Indicando con u l'ultima località del percorso corrente, il costo del successore è:

$$g' = g + \mathrm{dist}(u, v).$$

Ogni successore rappresenta un'alternativa distinta: gli aggiornamenti non modificano lo stato padre né gli altri successori.

1.3 Ordine di visita tramite DFS

Per determinare l'ordine di esplorazione degli stati, l'algoritmo utilizza una **visita in profondità** (DFS, *Depth-First Search*), implementata mediante una pila con politica LIFO: l'ultimo stato inserito è il primo a essere estratto. La DFS fornisce una procedura di base semplice da descrivere e da integrare con la potatura. L'Algoritmo 1 presenta esclusivamente la strategia di visita; il criterio di branch-and-bound viene descritto nella sottosezione successiva.

Algoritmo 1 Depth-First Search

Dati: Stato iniziale s_0
Risultato: Visita di tutti gli stati dell'albero in profondità

```
S ← pila vuota                                  ▷ Conserva gli stati in attesa
push(S, s_0)
while S non è vuota do
    s ← pop(S)                                  ▷ Estrae l'ultimo inserito
    visita s
    L ← Successori(s)                           ▷ Genera le estensioni ammissibili
    for s' ∈ L nell'ordine di generazione do
        push(S, s')
    end for
end while
```

La funzione Successori(s) applica la regola di espansione definita nella sottosezione precedente. Se non rimangono consegne da effettuare, restituisce una lista vuota. La baseline inserisce i successori nella pila nell'ordine prodotto dal generatore ed esplora per primo l'ultimo inserito. Nel codice, il generatore attraversa un `frozenset`: non viene applicato un ordinamento esplicito per indice e tale ordine non va assunto come garanzia della struttura dati. La visita termina quando la pila è vuota. L'operazione **visita** indica l'elaborazione dello stato estratto; nel caso di una foglia, si può valutare il giro completo a essa associato.

La DFS determina l'ordine di esplorazione, ma non riduce di per sé il numero di stati da esaminare. In assenza di potatura, il numero complessivo di stati attraversati è $\Theta(n!)$, dove n indica il numero di consegne, escluso il deposito. Il costo temporale effettivo dipende anche dalla rappresentazione e dalla generazione degli stati. Questa crescita fattoriale del numero di stati limita la scalabilità della ricerca esaustiva e motiva l'introduzione del branch-and-bound, che consente di eliminare i rami che non possono migliorare la migliore soluzione completa già trovata.

1.4 Eliminazione dei rami tramite branch-and-bound

Il branch-and-bound evita di approfondire i rami che non possono migliorare la migliore soluzione completa già trovata. Si mantiene il relativo percorso insieme al suo costo G_{best}, inizialmente posto a $+\infty$.

Quando viene raggiunto uno stato $s = (p, N, g)$ con $N = \emptyset$, si completa il percorso tornando al deposito. Indicando con u l'ultima località visitata, il costo del giro è:

$$G = g + \text{dist}(u, d).$$

Se $G < G_{\text{best}}$, si aggiorna il costo migliore e si conserva il giro corrispondente. Il valore G_{best} descrive quindi una soluzione effettivamente disponibile, non un ottimo già noto.

Per valutare se proseguire da uno stato si utilizza un limite inferiore $LB(s)$ al costo di qualsiasi giro che completi il suo percorso parziale. Nella versione di base si adotta:

$$LB(s) = g(s),$$

dove $g(s)$ è il costo accumulato nello stato. Questo limite è valido perché le distanze sono non negative: aggiungere altri spostamenti non può diminuire il costo già sostenuto.

Il controllo viene eseguito dopo l'estrazione dalla pila e prima dell'eventuale generazione dei successori. Se:

$$LB(s) \geq G_{\text{best}},$$

lo stato viene scartato senza espanderlo. Qualunque completamento avrebbe infatti un costo almeno pari a una soluzione già disponibile e non potrebbe migliorarla. La ricerca mira a trovare un giro ottimo, non a enumerare tutti i giri di costo ottimo.

Lo stesso criterio viene applicato anche prima di inserire un successore nella pila: se il suo costo accumulato è già almeno G_{best}, il successore viene scartato immediatamente. Il controllo dopo l'estrazione resta necessario perché il miglior costo completo può diminuire mentre altri stati sono ancora pendenti.

Il limite scelto è economico da calcolare, ma poco informativo, poiché non considera la distanza ancora necessaria per completare il giro. La potatura può ridurre gli stati esplorati nella pratica, ma la sua efficacia dipende dalle istanze e dalla qualità della soluzione corrente; non si assume un miglioramento della complessità nel caso peggiore.

1.5 Orizzonte, frontiera attiva e condizioni di arresto

L'*orizzonte della ricerca* descrive il confine al quale si arresta l'esplorazione esplicita corrente. Nel presente progetto si adotta come orizzonte H_t l'insieme delle foglie dell'albero esplicito generato fino al termine dell'iterazione t. Si tratta di un confine che evolve durante la ricerca: espandere una foglia la sostituisce con i suoi successori. Nella DFS tali foglie possono avere profondità diverse; l'orizzonte non coincide quindi con un unico livello dell'albero.

Per descrivere lo stato delle alternative si distinguono tre insiemi disgiunti:

- P_t: stati generati ancora pendenti nella pila;
- T_t: stati terminali già valutati, per i quali è stato calcolato il costo del giro completo, incluso il ritorno al deposito;
- D_t: stati esclusi mediante branch-and-bound, prima dell'inserimento nella pila oppure dopo l'estrazione.

Con questa convenzione, anche un ramo già risolto o potato contribuisce al confine dell'albero esplicito, pur non richiedendo altro calcolo:

$$H_t = P_t \cup T_t \cup D_t, \qquad |H_t| = |P_t| + |T_t| + |D_t|.$$

Questi insiemi descrivono l'albero costruito dalla ricerca; non indicano che tutti gli stati debbano essere conservati in memoria.

La *frontiera attiva* F_t comprende invece le alternative pendenti ancora ammissibili a ricevere ulteriore calcolo. Nella baseline, con il limite inferiore $LB(s) = g(s)$ e il miglior costo completo $G_{\text{best},t}$ disponibile, si definisce:

$$F_t = \{s \in P_t : g(s) < G_{\text{best},t}\}, \qquad F_t \subseteq P_t \subseteq H_t.$$

L'ammissibilità è valutata rispetto al controllo di potatura adottato: non afferma che lo stato conduca effettivamente a un miglioramento. Prima di trovare un giro completo si ha $G_{\text{best},t} = +\infty$.

La pila contiene quindi i candidati della frontiera attiva. Se nel frattempo viene trovato un giro migliore, alcuni stati in attesa possono non soddisfare più il criterio. Il controllo successivo al `pop` li scarta quando vengono estratti. Per questo il numero di stati pendenti può essere maggiore della dimensione della frontiera attiva. La politica LIFO stabilisce quale candidato esaminare per primo; il singolo stato estratto non coincide con l'intera frontiera.

L'implementazione registra il numero di stati pendenti (`pending_states`), quello degli stati ancora ammissibili (`active_frontier_size`), i giri completi valutati e le due categorie di potatura. La statistica `horizon_size` applica la somma precedente. Gli stati terminali e potati sono contabilizzati senza conservarne l'intero insieme. Anche a ricerca conclusa si può quindi avere $F_t = \emptyset$ e $H_t \neq \emptyset$.

La ricerca termina quando la pila è vuota oppure quando viene raggiunto un budget prefissato B. Un'unità di budget corrisponde all'estrazione e all'elaborazione di uno stato, anche se questo viene potato o è terminale. La generazione dei suoi successori rientra nella stessa unità. Il numero di estrazioni non coincide necessariamente con le espansioni effettive e non garantisce un tempo costante per passo.

Se la pila si svuota, tutte le alternative sono state valutate o escluse tramite un limite valido: il miglior giro trovato è ottimo. Questa è la condizione di certificazione usata dall'implementazione. Se il budget termina con stati ancora in attesa, si restituisce il miglior giro completo disponibile senza certificarne l'ottimalità. Qualora nessun giro completo sia stato trovato, viene segnalato tale esito, che non equivale all'assenza di soluzioni ammissibili.

Gli stati rimasti pendenti alla scadenza del budget rappresentano alternative irrisolte: l'interruzione non costituisce una potatura. Nella versione corrente vengono restituite soltanto la soluzione, l'indicazione di completamento e le statistiche; la pila locale non viene conservata e la ricerca non è riprendibile. La perdita delle alternative pendenti è un effetto dell'arresto e della restituzione del risultato, non una prova che i loro completamenti siano peggiori. La sospensione di un'alternativa deve essere distinta dalla potatura sicura del branch-and-bound, che esclude definitivamente un ramo perché non può migliorare la soluzione corrente.

2 Introduzione della previsione

Il componente appreso stima il *cost-to-go*: il costo minimo ancora necessario per visitare le consegne rimaste e tornare al deposito, condizionatamente al percorso parziale già compiuto. La previsione $\widehat{h}_\theta(s)$ non comprende il costo accumulato $g(s)$. Il modello riceve quattro feature dello stato; i target usati per addestrarlo sono descritti nella Sezione 3.

2.1 Feature e modello appreso

Indicando con u il nodo corrente e con d il deposito, lo stato viene trasformato nel vettore $x(s)$ di quattro feature:

$$x_1(s) = |N|, \qquad x_2(s) = \mathrm{dist}(u, d),$$
$$x_3(s) = \min_{v \in N} \mathrm{dist}(u, v), \qquad x_4(s) = \frac{1}{|N|} \sum_{v \in N} \mathrm{dist}(u, v).$$

Se $N = \emptyset$, si assegnano $x_3 = x_4 = 0$. Queste quantità descrivono quante consegne restano e alcune distanze utili a stimarne il costo. Non rappresentano l'intera geometria delle consegne: stati diversi possono avere le stesse feature e costi rimanenti differenti.

Si utilizza una regressione lineare di `scikit-learn`:

$$\widehat{h}_\theta(s) = b + \sum_{j=1}^{4} w_j x_j(s).$$

L'addestramento, effettuato tramite `LinearRegression.fit`, determina i pesi e l'intercetta minimizzando la somma dei residui quadratici sui dati di addestramento. La metrica di valutazione descritta sotto è invece l'errore assoluto medio. I parametri correnti, arrotondati solo qui per la presentazione, sono:

$$b = 42{,}322615,$$
$$(w_1, w_2, w_3, w_4) \simeq (12{,}192801,\ 0{,}305191,\ -1{,}284745,\ 2{,}194347).$$

Un peso negativo non stabilisce una relazione causale tra quella feature e il costo: i coefficienti sono stimati congiuntamente su feature correlate.

Questi parametri appartengono al modello principale su sei consegne. I valori numerici originali sono salvati in `data/model.json`, nei campi `weights` e `intercept`. La funzione di previsione può così applicare la formula dopo aver riletto i parametri, senza riaddestrare il modello. La corrispondenza con `model.predict` è stata verificata: sul primo esempio di test entrambe le modalità restituiscono circa 223,523423, con sole differenze di arrotondamento in virgola mobile. Il file `data/result.json` conserva invece le feature, i target e gli identificativi delle istanze; non è il file del modello addestrato.

Il dataset didattico utilizza la stessa definizione delle feature e la stessa procedura di regressione, ma produce un modello distinto in `data/didattico/model.json`. La sua intercetta è circa 2,025911 e i coefficienti sono (0,530833, 0,284551, −1,619936, 2,589103). Questo secondo addestramento non sostituisce i parametri del modello principale usato nel confronto sui seed 50–53.

3 Origine dei target

3.1 Ricerca insegnante e generazione degli esempi

Per uno stato $s = (p, N, g)$, il predittore stima il costo minimo ancora necessario per visitare le consegne in N e tornare al deposito, mantenendo fisso il percorso parziale p. Il target, indicato con $h^*(s)$, non comprende il costo $g(s)$ già sostenuto.

I target sono ottenuti con il branch-and-bound usato come insegnante. Per ogni stato campionato si avvia una ricerca da quello stesso stato, con budget di 100 000 estrazioni. Se la ricerca termina e restituisce un completamento ottimo di costo totale $G^*(s)$, si conserva:

$$h^*(s) = G^*(s) - g(s).$$

L'esempio viene escluso se la ricerca non termina o non restituisce un giro. Il target è dunque certificato dalla ricerca insegnante, non ricavato dalla previsione del modello. Il ritorno al deposito è incluso anche quando $N = \emptyset$.

Il dataset principale comprende dieci istanze, generate con seed da 42 a 51. Ogni istanza contiene sei consegne con coordinate uniformi in $[0, 100]^2$ e deposito fisso in $(0, 0)$. Si raccolgono gli stati dopo una, due, fino a sei consegne, senza includere lo stato iniziale. Per ciascuna istanza si ottengono

$$\sum_{k=1}^{6} \frac{6!}{(6-k)!} = 1\,956$$

esempi, per un totale di 19 560. La profondità massima limita gli stati raccolti; non tronca il completamento cercato dall'insegnante.

Ogni esempio conserva l'identificativo dell'istanza, lo stato e il target `target_cost_to_go`. Le coordinate sono associate all'istanza. Il file `data/examples.json` contiene questi dati; la matrice delle distanze viene ricostruita una volta per istanza e condivisa tra i suoi stati, poiché le coordinate non cambiano durante il percorso.

3.2 Separazione delle istanze

La separazione viene effettuata per istanza: i seed da 42 a 49 forniscono 15 648 esempi di addestramento; i seed 50 e 51 forniscono 3 912 esempi di verifica. Stati dello stesso problema non vengono distribuiti tra i due gruppi. Gli esempi derivati da una stessa istanza restano comunque correlati: il numero di esempi non equivale al numero di problemi indipendenti osservati.

I seed 52 e 53 sono generati successivamente soltanto per il confronto online: non fanno parte di questi 19 560 esempi e non modificano il modello. L'uso dei seed 50 e 51 nello sviluppo del regolatore è distinto dalla separazione usata per apprendere i pesi, come precisato nella Sezione 8.

3.3 Dataset didattico separato

Un secondo dataset contiene dieci mappe disegnate a mano, ciascuna con quattro consegne e deposito in $(0, 0)$. Le coordinate intere descrivono forme semplici, quali linea, croce, rettangolo e gruppi di punti: sono esempi artificiali in unità arbitrarie, non registrazioni di consegne reali. La raccolta usa lo stesso insegnante e comprende

$$\sum_{k=1}^{4} \frac{4!}{(4-k)!} = 64$$

stati per mappa, per un totale di 640 esempi. Le otto mappe con prefisso `train` forniscono 512 esempi. Le altre due mappe forniscono i 128 esempi di verifica e sono escluse dall'addestramento; sono identificate come `test_triangolo` e `test_irregolare`.

Per esempio, nella mappa in linea il deposito e le consegne hanno coordinate

$$(0, 0),\ (1, 0),\ (2, 0),\ (3, 0),\ (4, 0).$$

Lo stato con percorso $p = (0, 4)$ ha costo accumulato 4 e consegne residue $\{1, 2, 3\}$. Il completamento $4 \to 3 \to 2 \to 1 \to 0$ costa ancora 4: il target è $h^*(s) = 4$, mentre il costo totale del giro è 8. Il corrispondente vettore di feature è $x(s) = (3, 4, 1, 2)$.

Le mappe e gli esempi sono conservati in `data/didattico/instances.json` e `data/didattico/examples.json`. Anche il modello e le feature preparate sono salvati in questa cartella, separatamente dal modello principale. Tutti i 640 target sono stati controllati confrontandoli con l'enumerazione degli ordini possibili delle consegne residue.

4 Ricerca predittiva non regolata

La versione predittiva conserva pila, budget, condizioni di potatura e statistiche della baseline. Cambia l'ordine di esplorazione dei successori. Per uno stato candidato si calcola il costo totale stimato:

$$\widehat{f}(s) = g(s) + \widehat{h}_\theta(s).$$

Il costo già percorso deve essere incluso: confrontare soltanto il costo rimanente ignorerebbe la distanza già spesa per raggiungere lo stato. Se $N = \emptyset$, si usa direttamente $g(s) + \mathrm{dist}(u, d)$, senza interrogare il modello.

A ogni espansione i successori vengono ordinati per $\widehat{f}$ decrescente e inseriti nella pila in tale ordine. Poiché la pila è LIFO, il candidato con stima minore viene estratto per primo. Si tratta di un ordinamento locale dei successori: la ricerca rimane in profondità e non ordina globalmente l'intera frontiera come una coda a priorità.

La previsione non è un limite inferiore ammissibile e può sovrastimare il costo reale. Per questo viene usata soltanto per l'ordinamento; la potatura conserva il limite sicuro $LB(s) = g(s)$. Se la pila si svuota, anche la versione predittiva certifica l'ottimo. Con budget esaurito, restituisce il miglior giro disponibile senza tale certificazione.

Questa versione risponde alla domanda *quale alternativa sembra promettente?* mediante una regola fissa basata sul modello. Non contiene un regolatore che osservi l'andamento della ricerca e modifichi la politica di allocazione del calcolo: corrisponde alla chiamata con `patience=None`. La versione regolata è descritta di seguito.

5 Regolazione della ricerca predittiva

Il sistema C conserva il modello, l'ordinamento locale dei successori e la potatura del sistema B. Introduce una scelta adattiva dello stato da estrarre dalla pila, basata sull'assenza di miglioramenti del miglior giro completo trovato. Il regolatore è una regola esplicita, non un secondo modello appreso.

5.1 Osservazioni e decisione computazionale

Prima di ogni estrazione si osservano due contatori, inizializzati a zero:

- q_t: stati estratti dall'ultimo miglioramento del costo del miglior giro; aumenta a ogni estrazione e viene azzerato quando si trova un giro strettamente migliore, incluso il primo giro;
- c_t: estrazioni ordinarie effettuate dall'ultima diversificazione; aumenta solo quando si usa la normale estrazione LIFO e viene azzerato quando interviene il regolatore.

Prima del primo giro, q_t conta le estrazioni dall'inizio della ricerca. I conteggi comprendono anche stati terminali e stati poi potati. Con soglia $P = 100$, la condizione di intervento è

$$q_t \geq P \quad \wedge \quad c_t \geq P.$$

Il primo requisito segnala stagnazione; il secondo introduce una pausa di almeno 100 estrazioni ordinarie tra due interventi. Un miglioramento del giro azzera q_t, ma non c_t: i due contatori misurano eventi diversi.

Indicando con S_t la pila degli stati pendenti, si seleziona

$$s_t = \begin{cases} \arg\min_{s \in S_t} g(s), & q_t \geq P \ \wedge \ c_t \geq P, \\ \mathrm{ultimo}(S_t), & \text{altrimenti.} \end{cases}$$

Nella scansione del minimo, a parità di costo viene mantenuta la prima posizione incontrata. Il candidato selezionato viene rimosso dalla pila; gli altri rimangono disponibili. Dopo la selezione

si applicano le stesse condizioni di potatura e completamento del sistema B. In particolare, se uno stato pendente non è più attivo perché $g(s) \geq G_{\text{best}}$, viene potato al momento dell'estrazione. La diversificazione non elimina alternative e non introduce potature basate sulle previsioni.

La relazione richiesta dalla consegna è quindi

$$O_t = (q_t, c_t) \xrightarrow{R} U_t = \text{criterio di scelta del prossimo stato.}$$

Ogni scelta assegna la successiva unità del budget a un candidato. Il costo ottimo di riferimento non è un'osservazione del regolatore: resta riservato alla valutazione. Il contatore `diversification_steps` registra una unità per ogni estrazione decisa con il criterio del minimo costo parziale.

5.2 Motivazione e limiti della politica

La diversificazione cerca di interrompere una sequenza di esplorazione che non migliora il risultato, assegnando calcolo a un'alternativa con minor costo già sostenuto. Un costo parziale basso non garantisce però un buon completamento: può identificare uno stato poco profondo con molte consegne ancora da visitare. La pausa evita che il criterio sul costo parziale sostituisca continuamente quello predittivo.

La soglia è una scelta sperimentale e non un valore dimostrato ottimale. La stagnazione non misura direttamente l'errore del predittore né l'incertezza del singolo stato. La ricerca del minimo richiede una scansione di $|S_t|$ elementi a ogni intervento; il relativo costo non è rappresentato dal solo conteggio delle estrazioni.

6 Errore di previsione e necessità di ricerca

6.1 Misura dell'errore sui dati di verifica

La misura usata è l'errore assoluto medio, o MAE:

$$\text{MAE} = \frac{1}{m} \sum_{i=1}^{m} \left| \widehat{h}_\theta(s_i) - h^*(s_i) \right|.$$

Si sommano gli errori in valore assoluto affinché sovrastime e sottostime non si annullino. Il risultato è espresso nelle stesse unità delle distanze, non in percentuale. Come riferimento elementare si usa una previsione costante pari alla media dei target del solo insieme di addestramento.

Tabella 1: Errore del predittore sui 3 912 stati delle due istanze di verifica.

Metodo	MAE
Previsione costante	44,569
Regressione lineare	21,123

La riduzione del MAE rispetto alla previsione costante è circa il 52,61%. Questo risultato riguarda la stima del costo rimanente; non dimostra da solo che la ricerca guidata trovi giri migliori. La metrica aggrega stati a profondità diverse, con molti più esempi vicini al termine del giro rispetto agli stati iniziali. In particolare, include gli stati senza consegne rimaste; nella ricerca, invece, tali stati saranno valutati usando il costo esatto di ritorno al deposito.

Sul dataset didattico separato, il MAE dei 128 esempi di verifica è 1,318 per il modello lineare e 2,466 per la previsione costante, con una riduzione di circa il 46,54%. Anche qui la costante

è calcolata solo sui target di addestramento. La previsione sul primo esempio di verifica, prima e dopo la rilettura dei parametri salvati, è circa 7,603719 in entrambi i casi: questo controlla la coerenza del salvataggio, mentre l'accuratezza è misurata dal MAE. I due dataset hanno scale geometriche diverse, quindi i loro MAE assoluti non sono direttamente confrontabili.

6.2 Confronto con il risultato di ulteriore ricerca

Quando una ricerca aggiuntiva determina un completamento ottimo, il residuo della previsione è

$$r(s) = \big(G^*(s) - g(s)\big) - \widehat{h}_\theta(s).$$

Per esempio, sull'istanza con seed 51, lo stato con percorso $p = (0, 2)$ ha costo accumulato 83,703625. Il modello stima un costo rimanente di 212,672954, mentre la ricerca completa da quello stato restituisce un costo totale di 327,297587, cioè un costo rimanente ottimo di 243,593961. Si ottiene dunque un residuo positivo di circa 30,921007: il modello sottostima il costo necessario per completare quel percorso parziale.

Il confronto identifica l'errore dopo aver eseguito la ricerca aggiuntiva. Se questa viene interrotta, il miglior giro trovato fornisce un costo superiore o uguale all'ottimo e il residuo non è automaticamente l'errore esatto del predittore. Il MAE globale e il confronto ex post non costituiscono, da soli, una stima online del bisogno di ricerca di un nuovo stato. Il sistema esaminato non usa tali residui per modificare l'allocazione del calcolo e non comprende un secondo modello dell'errore. Il regolatore usa invece la stagnazione del miglior giro e la distanza dall'ultima diversificazione, come descritto nella Sezione 5. Questi segnali non equivalgono a una stima del residuo del singolo candidato.

7 Scelta probabilistica e massa irrisolta

Una distribuzione $\pi(a \mid s)$ deve assegnare probabilità alle alternative di un punto decisionale e avere un significato dichiarato, per esempio come preferenza di esplorazione. Se U_t è l'insieme delle alternative ancora irrisolte, la massa corrispondente è

$$\rho_t = \sum_{a \in U_t} \pi(a \mid s).$$

La mancata esplorazione di queste alternative non dimostra che siano inutili: rimane una parte della distribuzione senza una risoluzione tramite ricerca esplicita.

L'implementazione esaminata produce un ordinamento deterministico dei successori e non assegna una distribuzione alle alternative. Pertanto non calcola $\pi(a \mid s)$ né ρ_t e non soddisfa questa componente della consegna. Il numero di stati pendenti e la dimensione della frontiera non sono probabilità: senza una distribuzione definita non possono essere interpretati come massa irrisolta. Nessun risultato riportato viene quindi presentato come misura probabilistica.

8 Esperimento

8.1 Protocollo e separazione del costo offline

Sono confrontati A (DFS con branch-and-bound), B (ricerca con ordinamento predittivo) e C (ricerca predittiva con regolatore e soglia $P = 100$). Ogni esecuzione parte dal deposito, con percorso $p = (0)$, sei consegne da visitare e costo accumulato nullo. I tre metodi ricevono la

stessa matrice e lo stesso budget $B \in \{50, 100, 200, 500\}$ di stati estratti. Ogni prova riparte da zero: i budget maggiori non riprendono una pila lasciata dalle prove precedenti.

Il modello principale è quello addestrato sui seed 42–49. I seed 50 e 51, esclusi dall'addestramento dei pesi, sono stati usati anche per esaminare e modificare la politica di regolazione: vanno quindi considerati casi di sviluppo della politica. Sono state successivamente generate due nuove istanze, con seed 52 e 53, mantenendo fissi modello, soglia e regola. Queste istanze sono usate soltanto per la valutazione, senza generare ulteriori esempi di allenamento o riaddestrare il modello.

La generazione dei target e l'addestramento sono operazioni offline. Anche l'ottimo di riferimento è calcolato separatamente, una volta per istanza, con B&B e budget 100 000. La ricerca di riferimento termina in tutti e quattro i casi. I costi ottimi sono rispettivamente 288,252135, 309,557853, 259,896129 e 349,580289 per i seed 50, 51, 52 e 53. Servono soltanto per valutare i risultati: non inizializzano la migliore soluzione e non guidano i metodi online.

Il budget misura estrazioni, comprendendo stati terminali e stati poi potati, non soltanto espansioni. B e C richiedono anche valutazioni del modello; C esegue inoltre scansioni della pila durante la diversificazione. Lo stesso numero di estrazioni non implica uguale tempo di esecuzione. Non sono riportate misure temporali, quindi il confronto riguarda la qualità a parità di estrazioni e non dimostra un vantaggio di velocità.

8.2 Misure di qualità e risultati A/B/C

Per ogni esecuzione si registrano il miglior costo totale G_B, il percorso, il completamento della ricerca, le statistiche di frontiera e potatura e, per C, il numero di diversificazioni. La qualità è misurata anche tramite il divario percentuale dall'ottimo:

$$\mathrm{gap}_{\%} = 100\,\frac{G_B - G^*}{G^*}.$$

Il divario viene calcolato soltanto se è disponibile un giro e il riferimento è certificato. Il codice gestisce il caso di ottimo nullo evitando la divisione per zero.

Tabella 2: Costi totali su sei consegne, incluso il ritorno al deposito. A: baseline; B: predittiva; C: predittiva regolata. D_C conta le estrazioni di diversificazione.

Seed	Budget	Costo A	Costo B	Costo C	D_C
50	50	384,226	309,735	309,735	0
50	100	380,162	309,735	309,735	0
50	200	309,735	288,252	288,252	0
50	500	288,252	288,252	288,252	3
51	50	336,449	369,651	369,651	0
51	100	331,273	364,901	364,901	0
51	200	331,273	331,273	309,558	1
51	500	309,558	309,558	309,558	4
52	50	296,682	259,896	259,896	0
52	100	296,682	259,896	259,896	0
52	200	278,501	259,896	259,896	1
52	500	259,896	259,896	259,896	4
53	50	425,883	403,590	403,590	0
53	100	425,883	381,082	381,082	0
53	200	394,277	354,992	354,992	0
53	500	349,580	354,992	349,580	2

Le sedici righe rappresentano 48 esecuzioni. In tutte, il numero di stati estratti è pari al budget e la pila non è vuota all'arresto: `is_complete=False`. Sono comunque disponibili giri completi. Un costo uguale all'ottimo di riferimento identifica un giro ottimo, entro la precisione numerica, ma non una certificazione ottenuta dal metodo online: trovare l'ottimo e completare la ricerca sono eventi distinti.

Sul seed 51 a budget 200, A e B hanno gap 7,01%, mentre C raggiunge l'ottimo con una diversificazione. Sul seed 52, B trova l'ottimo già entro 50 estrazioni e C non migliora ulteriormente il costo. Sul seed 53, a budget 200 i gap di A e B sono rispettivamente 12,79% e 1,55%; C non è ancora intervenuto. A budget 500, B resta all'1,55%, mentre A e C raggiungono l'ottimo; C effettua due diversificazioni. In questo caso il guadagno della regolazione è rispetto a B, mentre il costo finale coincide con quello di A.

8.3 Controllo separato su istanze didattiche

Il modello didattico della Sezione 3.3 è stato verificato separatamente su mappe con quattro consegne, usando budget $\{5, 10, 20, 50\}$. Sulla mappa `test_irregolare`, l'ottimo di riferimento è 15,512156. A budget 5, A trova costo 19,196 (gap 23,75%) e B costo 17,997 (gap 16,02%). Entro 10 estrazioni B raggiunge l'ottimo; A resta a 19,196 anche a budget 20 e raggiunge l'ottimo entro 50. C coincide con B: con soglia 100 non interviene in queste prove. Questo controllo illustra l'effetto dell'ordinamento appreso, ma non costituisce una valutazione della regolazione né sostituisce l'esperimento principale su sei consegne.

9 Discussione

9.1 Guadagno dalla previsione e dalla regolazione

Il guadagno della previsione si misura confrontando B con A. Sui seed 50 e 52, B raggiunge l'ottimo con budget inferiori tra quelli provati. Sul seed 51, invece, peggiora il risultato a budget 50 e 100; sul seed 53 è migliore ai budget piccoli, ma a 500 rimane sopra l'ottimo che A trova. Un MAE inferiore a quello della previsione costante non implica quindi un miglioramento uniforme dell'ordine di ricerca.

Il guadagno aggiuntivo della regolazione si misura confrontando C con B. Sul seed 51, una diversificazione permette di raggiungere l'ottimo entro 200 estrazioni, contro il gap del 7,01% di B. Sul seed 53, due diversificazioni consentono a C di raggiungere l'ottimo entro 500 estrazioni, mentre B conserva un gap dell'1,55%. Sui seed 50 e 52 C non migliora il costo trovato da B ai budget osservati. Un intervento non equivale quindi a un guadagno: può avvenire quando il giro ottimo è già stato trovato, senza che la ricerca lo abbia certificato.

La politica effettua deviazioni distanziate anziché sostituire continuamente l'ordinamento predittivo con il minimo costo parziale. Il criterio può tuttavia favorire stati poco profondi: il costo già sostenuto non descrive quello ancora necessario. La stagnazione segnala un andamento della ricerca, ma non dimostra che il modello stia sbagliando né identifica quale candidato produrrà un miglioramento.

9.2 Limiti della valutazione

I seed 50 e 51 hanno contribuito allo sviluppo della politica e non sono un test indipendente delle scelte del regolatore. I seed 52 e 53 sono stati valutati mantenendo fisse tali scelte, ma costituiscono soltanto due nuovi casi della stessa dimensione e distribuzione dei dati di addestra-

mento. L'evidenza è preliminare: non dimostra un vantaggio generale o un comportamento su problemi più grandi. La soglia 100 non è stata selezionata tramite una ricerca sistematica con un insieme di validazione dedicato.

Le estrazioni hanno costi diversi tra i metodi: feature, previsioni, ordinamenti e scansioni della pila richiedono lavoro aggiuntivo. Senza misure temporali non si può concludere che C sia più veloce. Inoltre, la scelta del regolatore dipende dal modello, dalla distribuzione dei dati e dal budget. Il dataset didattico è separato e serve a verificare il procedimento su esempi leggibili; i suoi errori assoluti non sono direttamente confrontabili con quelli delle mappe di scala maggiore.

9.3 Alternative poco promettenti e utilità di ulteriore ricerca

Una previsione elevata può ritardare un candidato, ma non ne certifica l'inutilità: la potatura resta basata sul costo accumulato. La diversificazione può assegnare calcolo a candidati diversi da quelli che sarebbero estratti normalmente. Il risultato sul seed 53 mostra un beneficio del cambiamento di esplorazione rispetto a B.

I dati aggregati non registrano però, per ogni candidato, la previsione originaria e il miglioramento prodotto dal suo completamento. Non permettono quindi di identificare sistematicamente stati che apparivano poco interessanti ma richiedevano ulteriore ricerca: l'ultima domanda della consegna non è ancora verificata a livello di singolo stato. Analogamente, il sistema non definisce ancora una distribuzione sulle alternative né una massa probabilistica irrisolta. Questi aspetti restano limiti espliciti dell'implementazione, non risultati già misurati.

10 Materiale di consegna e riproducibilità

10.1 Esecuzione

Il punto di ingresso è `src/main.py`. La variabile `MODE` seleziona tre operazioni distinte:

- `generate`: crea gli esempi e i target con la ricerca insegnante; le coordinate provengono dal generatore casuale o dalle mappe didattiche, secondo la configurazione;
- `train`: legge il dataset, prepara le feature, separa le istanze, addestra e salva i parametri del modello;
- `compare`: legge dataset e modello esistenti, calcola i riferimenti e confronta A, B e C sui budget scelti.

Dalla radice del sottoprogetto si esegue `python3 -m src.main`. Per riprodurre l'esperimento principale si imposta `USE_TEACHING_DATA=False`, `START_PATH=(0,)` e si usano sei consegne, seed di generazione 42–51 e budget $\{50, 100, 200, 500\}$. L'addestramento esclude i seed 50 e 51; la modalità di confronto genera inoltre le mappe 52 e 53 senza riaddestrare. Il sistema B usa `patience=None`; il sistema C usa `patience=100` con la pausa descritta nella Sezione 5. I risultati sono salvati in `results/search_comparison.json`.

Per riprodurre il controllo didattico si imposta `USE_TEACHING_DATA=True`. La generazione legge `data/didattico/instances.json`; l'allenamento e il confronto usano modello e dati della medesima cartella. Per le prove riportate si scelgono budget $\{5, 10, 20, 50\}$. I risultati sono separati in `results/didattico/search_comparison.json`. Il numero di consegne è ricavato dalle mappe: `N_DELIVERIES` e `DATASET_SEEDS` configurano soltanto il generatore casuale.

Le esecuzioni riportate utilizzano Python 3.14.6 e `scikit-learn` 1.9.1. La dipendenza è elencata in `requirements.txt`.

10.2 Codice, dati e risultati

I file necessari alla riproduzione sono organizzati come segue:

- `src/algorithms/baseline.py`: ricerca DFS con B&B;
- `src/algorithms/predictor.py`: feature, addestramento, lettura dei parametri e previsione;
- `src/algorithms/predictive_search.py`: ricerca predittiva, selezione regolata e contatori degli interventi;
- `src/algorithms/regulator.py`: condizione di stagnazione;
- `src/experiments/generate_dataset.py`: istanze casuali riproducibili e produzione dei target tramite ricerca insegnante;
- `data/examples.json`, `data/result.json` e `data/model.json`: dati, feature e modello principali;
- `data/didattico/`: coordinate, dataset, feature e modello didattici, con le istruzioni del relativo `README.md`;
- `results/search_comparison.json`: costi, percorsi, riferimenti, gap e statistiche delle 48 esecuzioni principali;
- `results/didattico/search_comparison.json`: risultati del controllo separato sulle mappe con quattro consegne.

Gli identificativi con seed, il generatore e la configurazione rendono riproducibili anche le nuove mappe 52 e 53. La distribuzione probabilistica e la massa irrisolta non sono ancora implementate; il materiale non viene quindi presentato come esperimento completo di tutte le componenti della consegna. Il registro delle interazioni AI è riportato nell'Allegato A.

A Registro delle decisioni supportate da AI

Sono riportate cinque interazioni che hanno influito sul sistema. Le richieste sono sintetizzate; le motivazioni esplicitano il criterio tecnico delle scelte adottate.

1. **Modello del costo rimanente.** Richiesta: come addestrare un predittore a partire dalle feature e dai costi corretti. Proposta: usare una regressione lineare con `scikit-learn`, addestrata con `fit`. Esito: accettata e implementata. Motivazione: il modello consente di esaminare esplicitamente i quattro pesi e l'intercetta e di salvare una formula di previsione semplice.

2. **Separazione dei dati per istanza.** Richiesta: come preparare i dati per l'addestramento e la verifica. Proposta: conservare l'identificativo di ogni istanza e separare gli otto seed di addestramento dai due seed di test. Esito: accettata e implementata. Motivazione: stati dello stesso problema condividono le coordinate; distribuirli tra addestramento e test renderebbe meno significativo il controllo su istanze non usate per apprendere i pesi.

3. **Uso della previsione nella ricerca.** Richiesta: come passare dalla previsione di un singolo stato a una ricerca predittiva. Proposta: ordinare i successori tramite $g + \widehat{h}$, inserendoli in ordine decrescente nella pila LIFO, e mantenere la potatura sul solo costo accumulato. Esito: accettata e implementata in `predictiveSearch`. Motivazione: esplorare prima le stime più basse senza trattare una previsione appresa come limite inferiore certificato.

4. **Valutazione della qualità a budget limitato.** Richiesta: come interpretare il costo trovato quando la ricerca non è completa. Proposta: calcolare un ottimo di riferimento con una ricerca separata e riportare l'*optimality gap*. Esito: accettata e implementata; il punto di ingresso è stato riorganizzato in generazione, addestramento e confronto. Motivazione: distinguere la qualità del giro trovato dalla certificazione di ottimalità e confrontare i metodi sullo stesso budget online, senza usare l'ottimo per guidarli.

5. **Regolazione mediante stagnazione e diversificazione.** Richiesta: introdurre un regolatore e comprendere come cambiasse la scelta dello stato dalla pila. Proposta: dopo una sequenza senza miglioramenti, estrarre lo stato pendente con costo parziale minimo. Esito: proposta modificata dopo le prove sui seed 50 e 51, introducendo un secondo contatore e una pausa tra gli interventi; la versione con soglia 100 è stata poi valutata sui seed 52 e 53. Motivazione: intervenire continuamente può favorire stati poco profondi; distanziare le deviazioni conserva spazio per la ricerca guidata dal predittore. La separazione dei due contatori distingue un miglioramento del giro da un semplice cambio di esplorazione.

Riferimenti bibliografici

[1] Mark de Berg et al. *An ETH-Tight Exact Algorithm for Euclidean TSP*. 2018. arXiv: 1807.06933. URL: https://arxiv.org/abs/1807.06933 (visitato il 22/10/2026).

[2] OpenDSA. *Reduction of Hamiltonian Cycle to Traveling Salesman*. Materiale didattico CS201, Duke University, primavera 2025. 2025. URL: https://opendsa.cs.vt.edu/ODSA/Books/duke/cs201/spring-2025/MW/html/hamiltonianCycle_to_TSP.html (visitato il 22/10/2026).